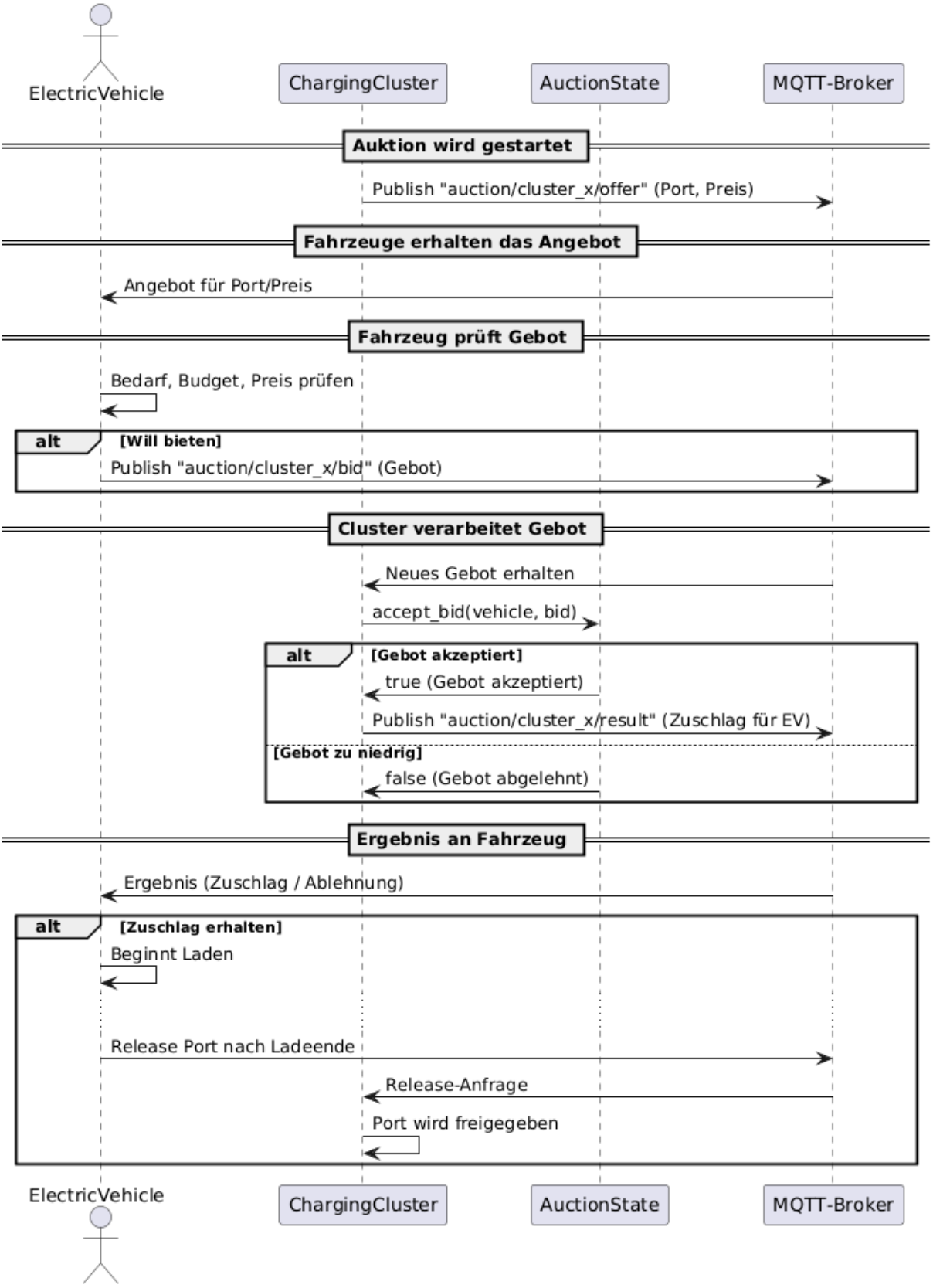
Bietprozess und Systemlogik im verteilten Ladesystem



1. Bietprozess im koordinierten (Auktions-)Modus

Im Auktionsmodus wird jeder freie Ladeport von einem Cluster als Auktion angeboten. Fahrzeuge können mitbieten und erhalten bei ausreichend hohem Gebot den Zuschlag.

Ablauf:

- Das Cluster startet eine Auktion und veröffentlicht das Angebot (Port, aktueller Preis) via MQTT.
- Fahrzeuge empfangen das Angebot, prüfen Bedarf, Budget und Preis.
- Fahrzeuge entscheiden, ob sie ein Gebot abgeben.
- Das Cluster prüft eingehende Gebote, vergibt ggf. den Zuschlag und informiert das Fahrzeug.
- Nach Ladeende meldet sich das Fahrzeug ab, der Port wird wieder freigegeben.

Codebeispiel: Auktion starten

```
def start_new_auctions(self):
    for port_id in available_ports:
        start_price = self.calculate_start_price()
        auction = AuctionState(port_id, self.cluster_id, start_price)
        self.active_auctions[port_id] = auction
        offer = {
            'cluster_id': self.cluster_id,
            'port_id': port_id,
            'current_price': start_price,
            'ticks_remaining': 5,
            'adaptive_mode': self.adaptive_mode
        }
        self.client.publish(
            f"auction/cluster_{self.cluster_id}/offer",
            json.dumps(offer)
        )
```

2. Entscheidungslogik bei Fahrzeugen (Clusterfindung und Bid-Abgabe)

Jedes Fahrzeug prüft eigenständig, ob es laden muss und ob ein Angebot attraktiv genug ist.
Entscheidungskriterien sind:

- Bedarf: Ist der Akku (SOC) niedrig oder die Deadline bald erreicht?
- Budget: Liegt der Gesamtpreis (Preis × Energiebedarf) im verfügbaren Budget?
- Clusterwahl: Bevorzugt werden Cluster, bei denen zuletzt keine Ablehnung erfolgte („Blacklist“-Mechanismus).

Codebeispiel: Bewertung und Bid-Entscheidung

```
def evaluate_auction_offer(self, cluster_id, port_id, price,
simulation_time):
    if not self.adaptive_mode or cluster_id in self.blacklisted_clusters or
self.active_auction:
        return False
```

```
    current_budget = self.calculate_current_budget(simulation_time)
    energy_needed = (self.target_soc - self.current_soc) * self.capacity
    total_cost = price * energy_needed
    return total_cost <= current_budget
```

Blacklist-Behandlung

```
def reset_auction_state(self):
    if self.active_auction:
        self.blacklisted_clusters.add(self.active_auction)
    self.active_auction = None
    self.active_port = None
    self.auction_timeout = 0
```

3. Preisfindung bei den Clustern

Der Preis eines Ladeangebots wird dynamisch bestimmt, abhängig von:

- **Auslastung des Clusters** (je weniger Ports frei, desto teurer)
- **Netzlast** (Strommangel = teurer, Stromüberschuss = günstiger)

Codebeispiel: Startpreis-Berechnung

```
def calculate_start_price(self):
    available_ports = self.get_available_ports()
    total_ports = len(self.ports)
    utilization_factor = 1.0 + (total_ports - available_ports) /
total_ports
    grid_factor = 1.0
    if self.grid_monitor.is_grid_stressed():
        grid_factor = 1.5
    elif self.grid_monitor.get_balance() > 0:
        grid_factor = 0.8
    start_price = self.base_price * utilization_factor * grid_factor
    return min(3.0, max(0.30, start_price))
```

Dynamische Preisänderung während der Auktion

```
def update_price(self, price_adjustment_factor):
    if not self.is_active:
        return
    self.current_price *= price_adjustment_factor
    self.current_price = max(0.50, min(50.0, self.current_price))
```

4. Buffer-Mechanismus der Cluster

Jedes Cluster besitzt einen Energiespeicher (Buffer), der den Ladebetrieb absichert. Sinkt der Buffer zu stark, werden keine neuen Auktionen mehr gestartet.

Codebeispiel: Buffer-Update

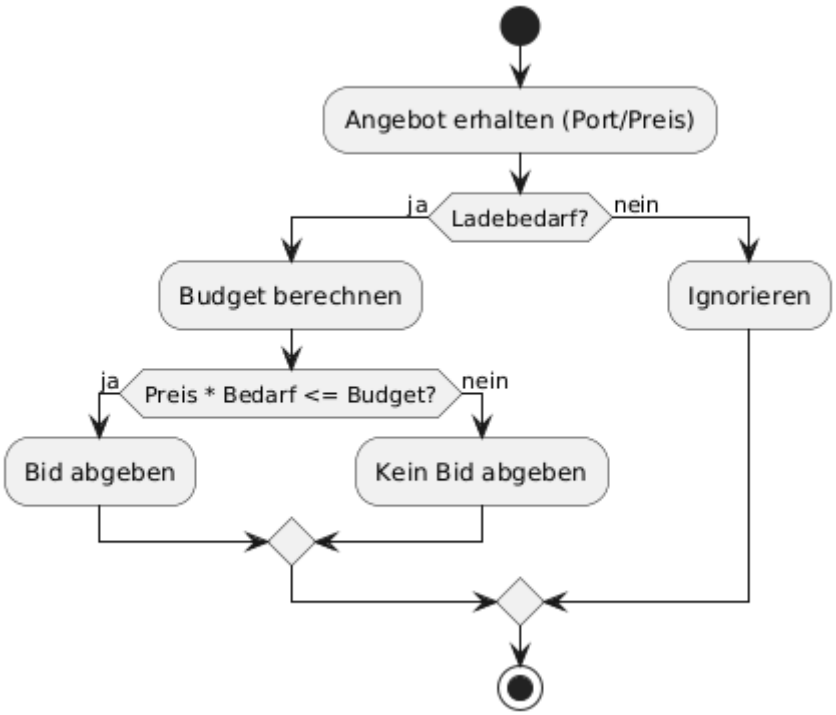
```
def update_buffer(self):
    current_power_usage = sum(port['power'] for port in
self.ports.values())
    if current_power_usage > 0:
```

```
        self.buffer_level = max(0, self.buffer_level - (current_power_usage
* 0.02))
    else:
        self.buffer_level = min(self.buffer_capacity, self.buffer_level +
1.0)
```

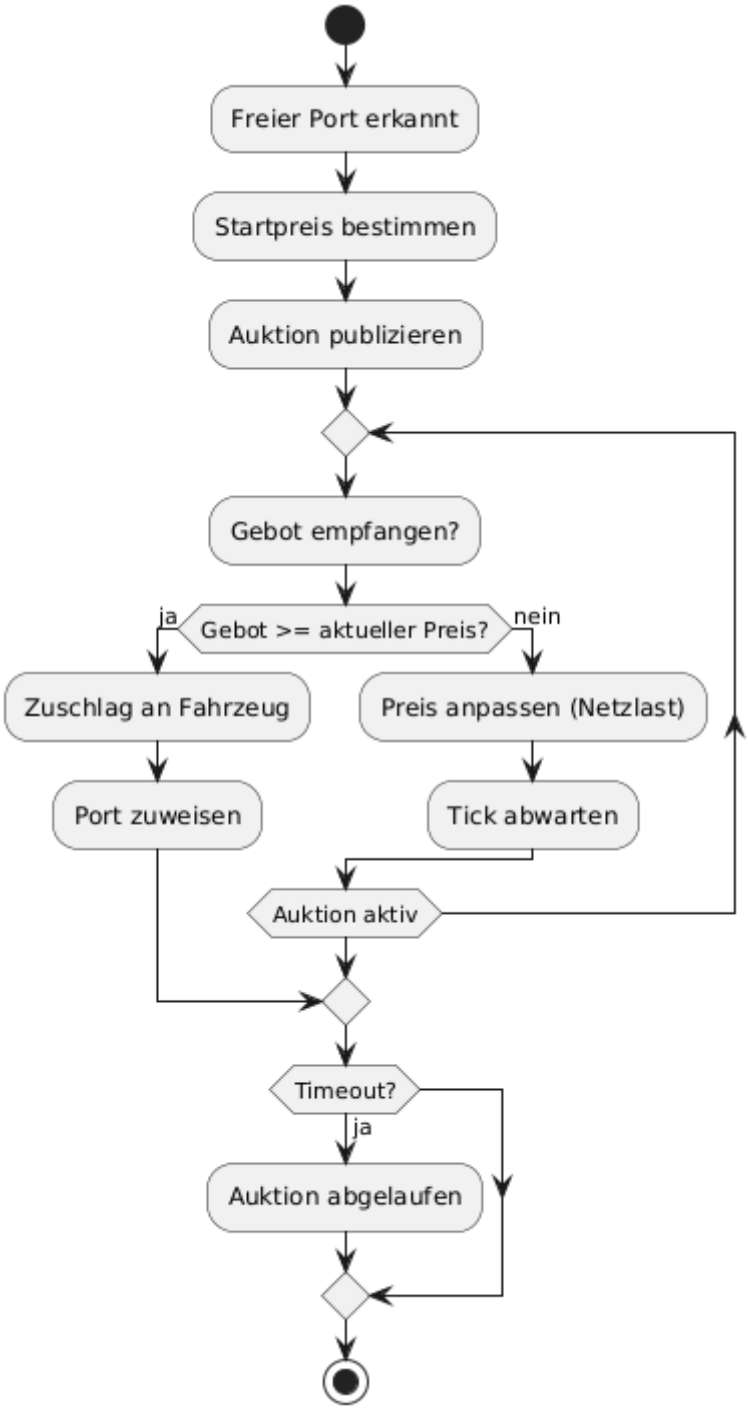
Schutzfunktion:

```
def start_new_auctions(self):
    if self.buffer_level < 10:
        return  # Keine Auktionen bei niedrigem Buffer
    # Auktionen starten ...
```

Flowchart 1: Entscheidungsprozess eines Fahrzeugs



Flowchart 2: Ablauf im Cluster (Auktion)



Zusammenfassung

Das System nutzt ein intelligentes, dezentrales Bietsystem für das Laden von Elektrofahrzeugen.
Die Entscheidung, wo und zu welchem Preis geladen wird, erfolgt durch:

- Flexible Preisgestaltung auf Clusterebene,
- Bedarfsgesteuerte, budgetbewusste Fahrzeuglogik,
- Sichere Steuerung des Energieflusses über Buffer.

Alle Systemkomponenten sind lose gekoppelt und kommunizieren asynchron per MQTT.